

Reverse Engineering Laboratory Assignment 1 Report

Reverse Engineering of Black-Box Machine-Learning Models

DAKSH RATHI B25132

1 Introduction

This report presents the reverse engineering of three supplied machine-learning systems: two three-feature linear regressors, a two-feature logistic classifier, and a neural network for zero-versus-non-zero MNIST classification. The experiments use model queries to reconstruct or characterize the systems, then validate the inferred behavior through numerical checks and individual plots. The discussion follows the submitted implementation and focuses on the functions that directly produce the required analysis.

2 Problem 1: Linear Regression Model Recovery

2.1 Objective and Overview

Each supplied regressor accepts three features and returns a continuous prediction through `predict()`. The working model is

$$f(x) = b + w_1x_1 + w_2x_2 + w_3x_3, \quad (1)$$

where the intercept b and coefficients w_1, w_2, w_3 are unknown. The task is to recover these parameters only from input-output queries, verify that the reconstructed equation reproduces the black box, test for non-linearity or pairwise interaction, and visualize the agreement.

2.2 Functions Required

Function	Role in the solution
<code>recover_intercept()</code>	Finds b from the model output at the origin.
<code>recover_coefficients()</code>	Finds w_1, w_2, w_3 using one-feature-at-a-time unit probes.
<code>estimate_model_error()</code>	Measures black-box versus reconstructed prediction error on random samples.
<code>detect_nonlinearity()</code>	Checks changing slopes and pairwise interaction residuals.
<code>plot_model_behaviour()</code>	Compares black-box and recovered predictions on a 3-D input grid.

2.3 Function-Wise Approach

`recover_intercept()`. The intercept is recovered with the smallest possible black-box experiment: a single query at $X = [0, 0, 0]$. For an affine model, all three feature contributions vanish at the origin and therefore

$$f(0, 0, 0) = b. \quad (2)$$

The code sends the origin in the two-dimensional shape expected by `predict()` and converts the one-element prediction to a scalar. This step requires no knowledge of the model coefficients or training data; it uses only the observable input-output behaviour of the supplied estimator.

`recover_coefficients()`. After obtaining the baseline response at the origin, the function probes the three standard unit vectors e_1, e_2, e_3 . The four queried outputs are $b, b + w_1, b + w_2$, and $b + w_3$. Subtracting the common baseline cancels the intercept and isolates one coefficient at a time:

$$w_i = f(e_i) - f(0), \quad i = 1, 2, 3. \quad (3)$$

This is essentially a finite-difference experiment with unit step size. Because only one coordinate is changed in each query, the measured output change can be assigned to that feature under the affine assumption. The implementation performs all four predictions together, flattens the returned array, and returns the recovered parameter vector with shape $(3,)$. Thus the complete three-feature model can be reconstructed from only four carefully chosen black-box queries.

`estimate_model_error()`. Parameter recovery is next checked away from the special probe points. With random seed 42, the function generates 100 reproducible inputs uniformly from $[-1, 1]^3$. It obtains the true black-box prediction vector and independently forms the reconstructed vector using $\hat{y} = b + Xw$. Their mean squared difference measures how accurately the recovered equation reproduces the supplied model over an unseen set of probes. The function also compares the variances of the two prediction vectors, which checks whether both models show the same spread of outputs across the sampled region. A near-zero recovery MSE together with matching variances is stronger evidence than checking only the four recovery points. This quantity is strictly a *model-recovery* error, not the original training or test error, since no ground-truth target values are used.

`detect_nonlinearity()`. The helper `predict_one()` reshapes a single probe into the form required by `predict()` and returns one scalar output. The main function then performs two complementary finite-difference tests. First, for each feature it compares the output increment from $0 \rightarrow 1$ with the increment from $1 \rightarrow 2$ through $f(2e_i) - f(e_i) - w_i$. A linear feature must have constant slope, so a non-zero residual would suggest curvature such as a quadratic or another transformed feature. Second, every pair (i, j) is tested using

$$f(e_i + e_j) - f(e_i) - f(e_j) + f(0). \quad (4)$$

For a purely additive affine model this mixed difference cancels exactly. A residual larger than the 10^{-6} tolerance would therefore indicate that the effect of one feature depends on the value of another. The function reports the offending features, interacting pairs, and a final Boolean non-linearity flag.

`plot_model_behaviour()`. The visualization provides an independent qualitative check of the numerical recovery. Ten evenly spaced values are selected for each feature in $[-1, 1]$, `meshgrid()` forms every coordinate combination, and `column_stack()` converts the grid into $10^3 = 1000$ three-feature samples. These same coordinates are passed to the black box and to the recovered equation. In each 3-D scatter plot the spatial position represents (x_1, x_2, x_3) while colour represents the predicted response, so matching colour patterns indicate that both models assign essentially the same output throughout the sampled volume rather than only at isolated points.

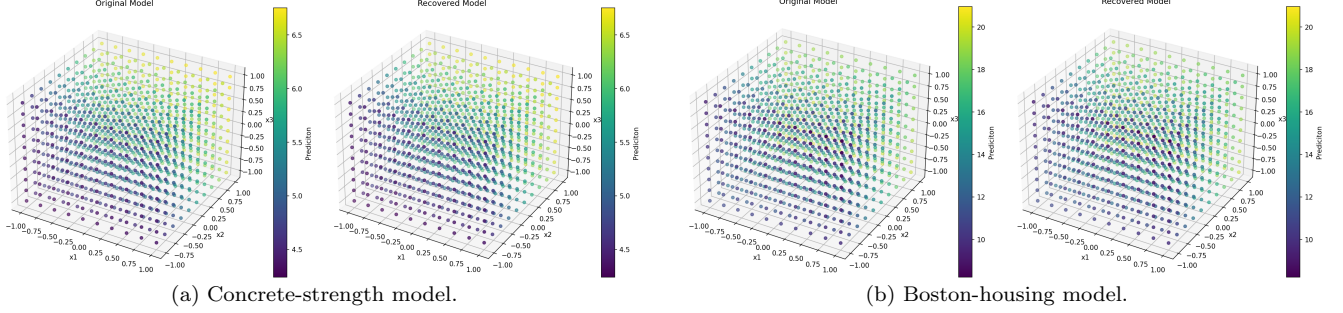


Figure 1: Black-box and reconstructed behavior for the two supplied regressors.

2.4 Observed Results

Model	b	w_1	w_2	w_3	Recovery MSE
Concrete	5.4959	0.0677	1.0869	0.1002	1.26×10^{-31}
Boston	14.5881	-0.5671	4.9331	-0.8671	3.25×10^{-30}

The black-box and recovered prediction variances are respectively 0.4736424505 and 0.4736424505 for Concrete, and 9.6966579582 and 9.6966579582 for Boston. No nonlinear feature or interacting pair is detected. Thus both regressors are reproduced to floating-point precision, with Feature 2 having the largest coefficient magnitude in both models.

3 Problem 2: Logistic Decision-Boundary Discovery

3.1 Objective and Overview

The second black box is a binary classifier with two features and exposes `predict()` and `predict_proba()`. The assumed form is

$$P(y = 1 | x) = \sigma(b + w_1x_1 + w_2x_2), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (5)$$

The task is to discover the decision boundary, recover the logistic parameters, compare feature influence, detect interactions, and test the sensitivity of points close to the boundary.

3.2 Functions Required

Function	Role in the solution
<code>class1_prob()</code> , <code>prob_to_logit()</code> , <code>sigmoid()</code>	Probability/logit helper functions used for recovery.
<code>plot_decision_boundary()</code>	Maps the classifier over a dense grid and draws $P(y = 1) = 0.5$.
<code>recover_coefficients()</code>	Recovers b, w_1, w_2 from three probability queries.
<code>compute_feature_importance()</code>	Measures probability sensitivity to one-feature perturbations.
<code>interaction_detection()</code>	Tests whether the logit contains non-additive feature effects.
<code>test_robustness()</code>	Pushes near-boundary points across the recovered hyperplane.
<code>plot_confidence_heatmap()</code>	Shows confidence throughout the input region.
<code>plot_coefficient_recovery()</code> , <code>evaluate_recovery()</code>	Validate recovered parameters and class agreement.

3.3 Function-Wise Approach

Probability/logit helpers and `recover_coefficients()`. `class1_prob()` standardizes model access by converting the input to a floating-point array and returning column 1 of `predict_proba()`, i.e. $P(y = 1)$. Logistic probabilities are nonlinear in the input, so `prob_to_logit()` applies the inverse-sigmoid transformation $L(p) = \log(p/(1 - p))$. Before this operation the probabilities are clipped to $[10^{-10}, 1 - 10^{-10}]$ so that values numerically equal to 0 or 1 do not produce infinite logits. Conversely, `sigmoid()` maps a recovered logit back to probability space.

The key recovery idea is that a logistic-regression model becomes affine after the logit transform:

$$L(P(y = 1 | x)) = b + w_1x_1 + w_2x_2. \quad (6)$$

Therefore only the three points $(0, 0)$, $(1, 0)$ and $(0, 1)$ are needed. Their logits directly give

$$b = L(p_{00}), \quad w_1 = L(p_{10}) - b, \quad w_2 = L(p_{01}) - b. \quad (7)$$

After recovery, the code evaluates 40,000 grid points, reconstructs their logits with $b + Xw$, applies `sigmoid()`, and compares these probabilities and classes with the black box. Thus the same function both recovers the parameters and numerically verifies that the recovered decision rule generalizes beyond the three probe points.

`plot_decision_boundary()`. Two 200-point axes over $[-5, 5]$ are expanded with `meshgrid()` and flattened into 40,000 input pairs. `predict()` supplies the discrete class regions, while `class1_prob()` supplies a continuous probability at every grid point. The class array is reshaped back to the two-dimensional grid for the filled regions, and the probability array is reshaped for the contour plot. Since $\sigma(z) = 0.5$ precisely when $z = 0$, the red $P(y = 1) = 0.5$ contour represents the geometric decision boundary $b + w^T x = 0$. The scatter colouring additionally shows how quickly the probability changes on either side of the boundary.

`compute_feature_importance()`. This function measures sensitivity rather than reading the model weights directly. Five hundred reproducible points are sampled in the specified feature ranges. Feature j is then increased by 5% of its total range while the other coordinate is held fixed; for $[-5, 5]$ this gives the same perturbation magnitude, $\Delta = 0.5$, for both features. Importance is the average absolute probability response

$$I_j = \frac{1}{n} \sum_i |P(X_i + 0.5e_j) - P(X_i)|. \quad (8)$$

Using absolute changes prevents increases and decreases from cancelling, while using the same relative step makes the two features comparable. Finally, the two raw scores are normalized to sum to one and the larger score is reported as the more influential feature over the sampled region.

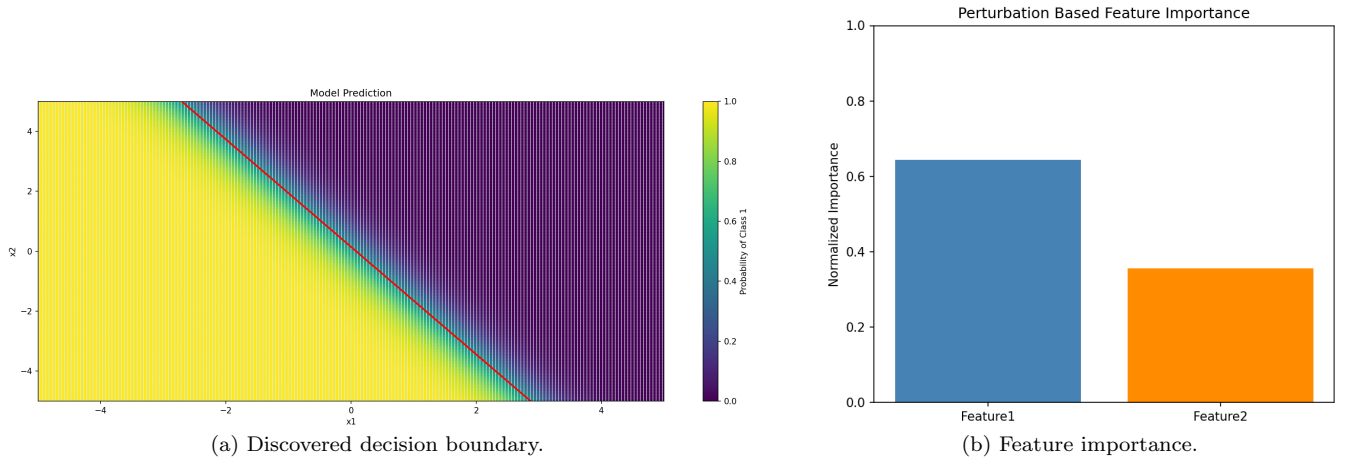


Figure 2: Decision-surface and sensitivity analysis.

`interaction_detection()`. A sigmoid output is nonlinear even when its underlying logistic model is perfectly additive, so interactions are tested in *logit space* rather than directly on probabilities. One hundred random points from $[-2, 2]^2$ are evaluated after four related perturbations: no change, $e_1 = (0.5, 0)$, $e_2 = (0, 0.5)$, and both perturbations together. The mixed finite difference

$$L(X + e_1 + e_2) - L(X + e_1) - L(X + e_2) + L(X) \quad (9)$$

removes the two individual feature effects. For the additive logit $b + w_1x_1 + w_2x_2$ the expression is identically zero, whereas an interaction term would leave a residual. The function reports both the mean and maximum absolute residual and compares the maximum with a 10^{-6} tolerance.

`test_robustness()`. Robustness is examined specifically where the classifier should be most sensitive. A dense grid is queried and the 100 samples with probabilities closest to 0.5 are selected. For the recovered hyperplane, the signed logit determines which side of the boundary a point occupies, while the perpendicular distance to the line is

$$d = \frac{|b + x^T w|}{\|w\|_2}. \quad (10)$$

The code forms a unit direction opposite to the current side of the boundary and moves each point by $d + 10^{-3}$. The extra 10^{-3} ensures that the point passes slightly beyond the line rather than landing numerically on it. Original and perturbed classes are then compared, and the flip rate and perturbation norm quantify local boundary sensitivity. `plot_confidence_heatmap()`. The model confidence is defined as $\max(p, 1 - p)$. It equals 0.5 at the decision boundary and approaches 1 as the classifier becomes more certain. The function evaluates the same dense two-feature region, reshapes confidence values to the grid, and overlays the $P(y = 1) = 0.5$ contour. Consequently, the heatmap gives a direct visual relation between distance from the boundary and prediction certainty.

Recovery-validation functions. `plot_coefficient_recovery()` places the supplied coefficients and the recovered coefficients side by side and computes their Euclidean difference. `evaluate_recovery()` additionally samples 1,000 fresh points, predicts their black-box classes, reconstructs classes from the recovered equation, and reports agreement. Direct access to `model.coef_` is used only at this final validation stage to compare “true versus recovered” coefficients; it is not used by the reverse-engineering procedure itself.

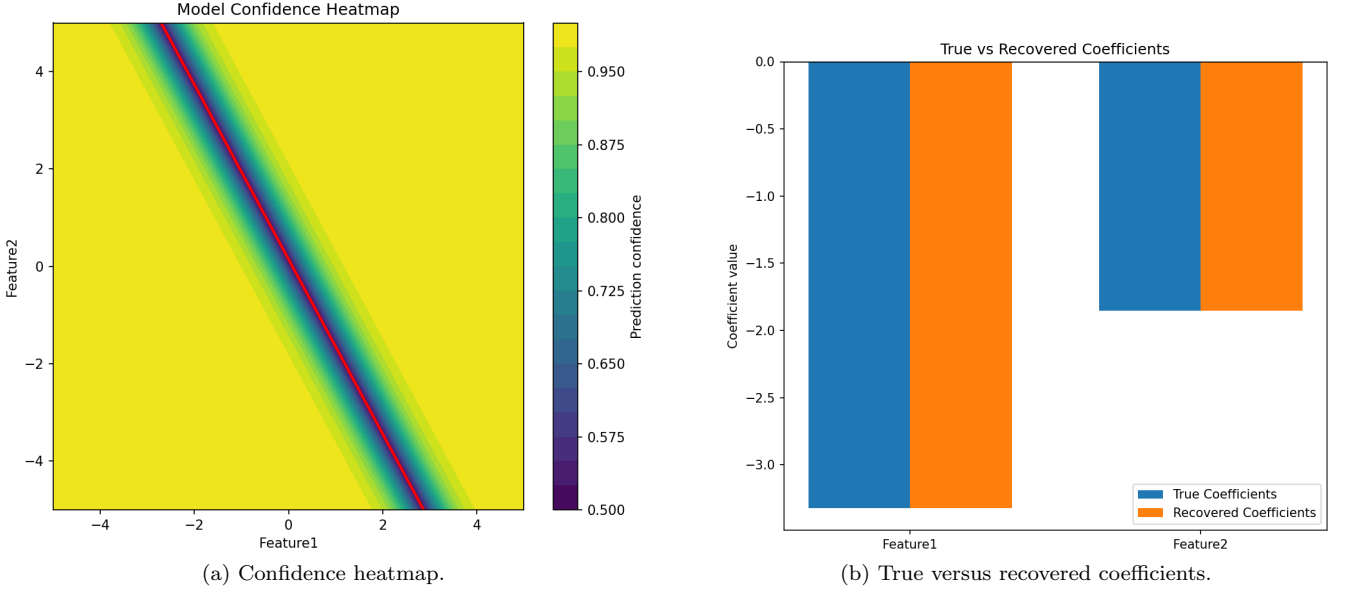


Figure 3: Confidence and final recovery validation.

3.4 Observed Results

The recovered classifier is

$$b = 0.2748229305, \quad w = [-3.3223869, -1.8519834]. \quad (11)$$

Probability MSE is 3.16179×10^{-33} and decision-boundary accuracy on the dense grid is 1.0. Raw feature importance $[0.04633039, 0.02569495]$ becomes $[0.64325127, 0.35674873]$ after normalization, making Feature 1 more influential under the common perturbation. Maximum/mean interaction scores are 7.2831×10^{-13} and 2.0721×10^{-14} , so no meaningful interaction is detected. All 100 selected near-boundary points flip class with mean perturbation norm 0.00651438. Final coefficient error is 0.0000 to four decimal places and class agreement is 1.0000.

4 Problem 3: Neural-Network Behaviour Analysis

4.1 Objective and Data Preparation

The supplied network classifies MNIST images as zero versus non-zero. The verified architecture is

$$20 \text{ inputs} \rightarrow 8 \text{ ReLU} \rightarrow 4 \text{ ReLU} \rightarrow 1 \text{ sigmoid}. \quad (12)$$

Images are flattened, normalized, passed through the supplied `StandardScaler`, and then reduced by PCA to 20 features. Class 1 represents digit zero and class 0 represents non-zero digits. The mapping gives full test accuracy 0.9946. A balanced set of 500 zero and 500 non-zero examples is used for the detailed analyses.

4.2 Functions Required

Function	Role in the solution
<code>get_probability()</code> , <code>predict_class()</code>	Return sigmoid probabilities and binary predictions.
<code>get_layer_output()</code>	Extracts activations from a chosen dense hidden layer.
<code>compute_input_importance()</code>	Uses feature permutation to rank the 20 PCA inputs.
<code>analyze_layer()</code>	Computes activation, dead-neuron, redundancy and class-separation statistics.
<code>compute_tsne_projection()</code>	Projects second-layer activations into two dimensions.
<code>detect_feature_interactions()</code>	Scores all 190 PCA-feature pairs for non-additive effects.
<code>compute_neuron_contributions()</code>	Ranks second-layer neurons by class-wise activation difference.
<code>layer_pruning_analysis()</code>	Masks low-contribution neurons and measures remaining accuracy.

4.3 Function-Wise Approach

Output and hidden-layer helpers. `get_probability()` is the common interface to the network output. Keras returns sigmoid predictions with shape $(n, 1)$, so the function converts them to a floating-point NumPy array and flattens them to $(n,)$. `predict_class()` applies the natural binary threshold of 0.5, producing class 1 for predicted zero and class 0 otherwise. To inspect the network internally, `get_layer_output()` first selects the dense layers from the loaded model and then builds an intermediate Keras model whose input is unchanged but whose output is the requested hidden layer. This gives activation matrices of shape $(n, 8)$ for hidden layer 1 and $(n, 4)$ for hidden layer 2 without modifying the original model.

compute_input_importance(). The 20 model inputs are PCA components rather than raw image pixels, so importance is measured at the PCA-feature level. The baseline probabilities are first stored. Then, for one feature at a time, that column is randomly permuted across the balanced samples while all other columns remain untouched. Permutation preserves the marginal distribution of the chosen feature but breaks its correspondence with the correct

sample and with the remaining feature pattern. If the model depends strongly on that feature, this disruption causes a larger probability change. The score

$$I_j = \frac{1}{n} \sum_i |P(X_i^{(j)}) - P(X_i)| \quad (13)$$

is computed for every component, normalized by the sum of all scores, and sorted to obtain the top-10 ranking. Thus the method measures behavioural dependence on each transformed input without requiring access to network weights. **analyze_layer()**. This function characterizes both individual neurons and the hidden representation as a whole. For every neuron it calculates the mean and standard deviation of activation, together with the fraction of values whose magnitude is below 10^{-10} . Since the hidden layers use ReLU, a neuron with at least 95% near-zero outputs is marked as dead. Pairwise Pearson correlations between neuron activation vectors are also computed; pairs with $|\rho| \geq 0.95$ are treated as potentially redundant because they respond almost identically across samples.

Class-specific means are then used to quantify discrimination. The implemented separation score is normalized by the neuron’s activation spread,

$$S_j = \frac{|\mu_{1,j} - \mu_{0,j}|}{\sigma_j + 10^{-10}}, \quad (14)$$

so a mean difference is interpreted relative to the natural variability of that neuron. In addition, the silhouette score evaluates whether the complete hidden representation clusters class 0 and class 1 separately. Finally, correlations between each centered PCA input and each centered neuron activation identify the input component most strongly associated with every neuron. Together these statistics reveal activity, redundancy, class separation, and input-to-neuron relationships in one analysis.

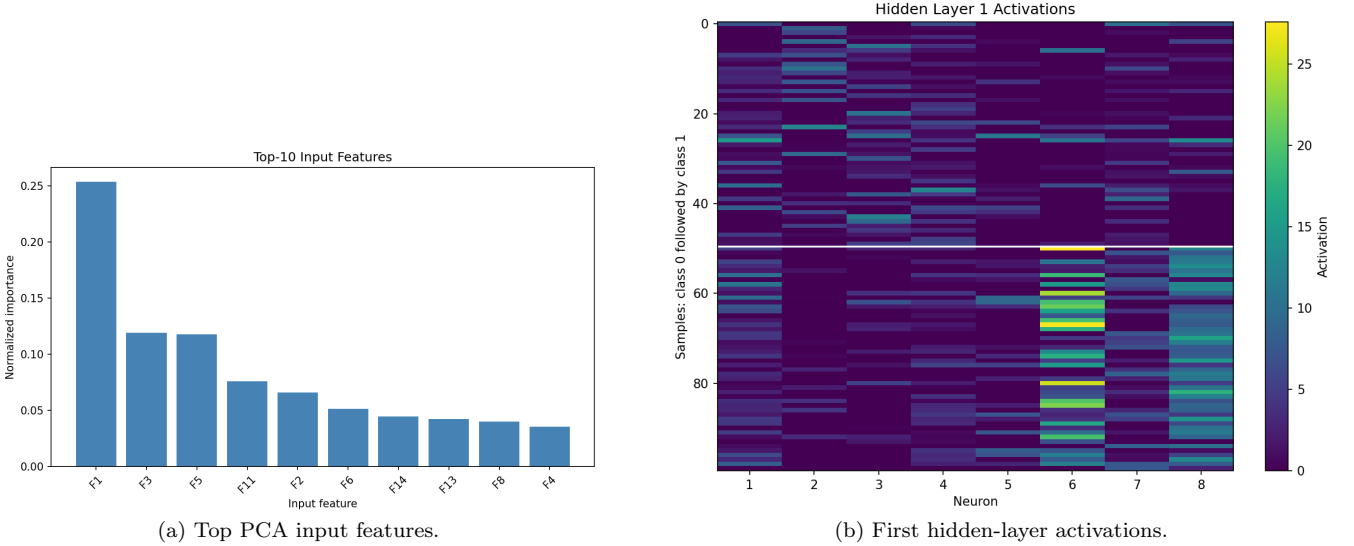


Figure 4: Input sensitivity and first-layer activation analysis.

compute_tsne_projection(). The second hidden layer contains only four activation coordinates, but those coordinates are still difficult to interpret directly. The function therefore projects this four-dimensional representation to two dimensions with t-SNE, using a fixed seed for reproducibility, PCA initialization, automatic learning rate and at most 1,000 samples. t-SNE is used here only as a visualization tool: it preserves local neighbourhood relationships sufficiently to show whether zero and non-zero samples occupy visibly different regions. The quantitative conclusion about class separation is taken from the silhouette analysis rather than from the appearance of the t-SNE plot alone. **detect_feature_interactions()**. With 20 PCA inputs there are $\binom{20}{2} = 190$ unique feature pairs, so the implementation restricts the experiment to at most 300 samples for efficiency. A reproducible permutation is prepared for every feature. The function first records the probability response when each feature is permuted individually, then permutes each pair together and evaluates the combined response. For pair (i, j) it computes

$$D_{ij} = \frac{1}{n} \sum_k |f(X_{ij,k}) - f(X_{i,k}) - f(X_{j,k}) + f(X_k)|. \quad (15)$$

If the effect of disturbing two features were simply the sum of their separate effects, the residual would be close to zero. Larger scores therefore indicate non-additive behaviour learned by the neural network. Scores are stored symmetrically in a 20×20 interaction matrix and the ten strongest pairs are returned for interpretation.

compute_neuron_contributions(). For each neuron in the second hidden layer, the function compares its average activation on zero examples with its average activation on non-zero examples. The raw contribution

$$C_j = |\mu_{1,j} - \mu_{0,j}| \quad (16)$$

is large when a neuron responds very differently to the two classes and is small when its average response is similar. The four scores are normalized to sum to one and sorted in descending order. This is a deliberately interpretable

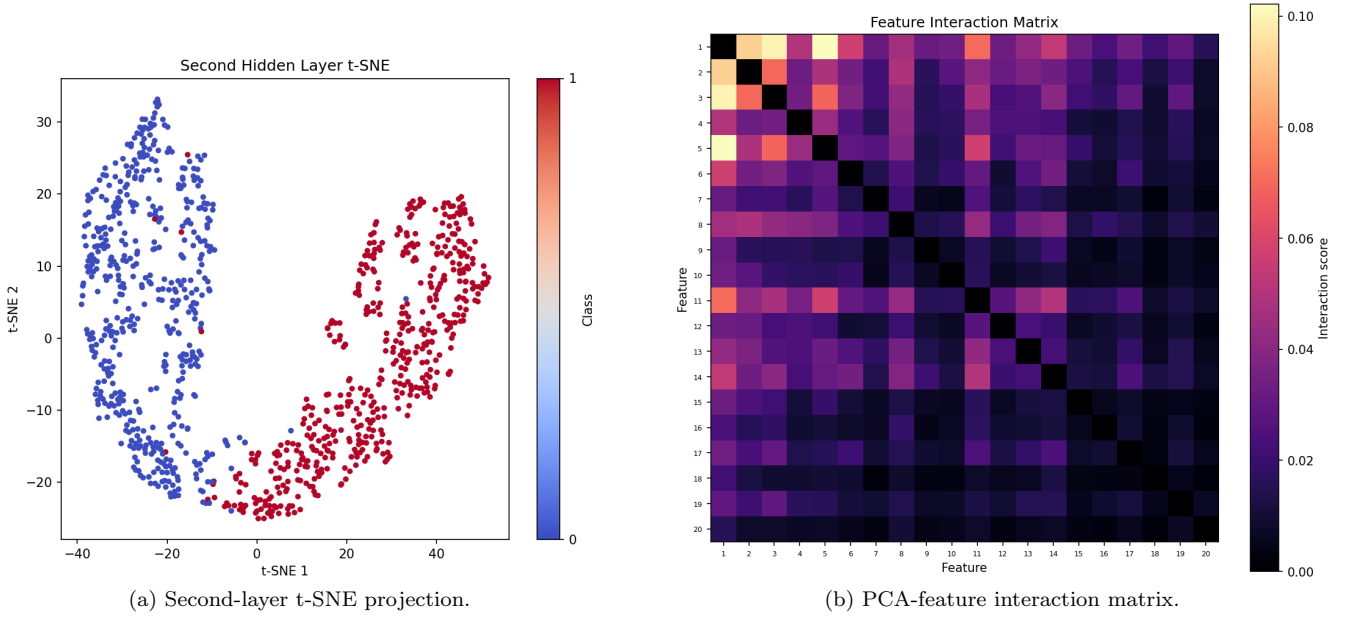


Figure 5: Representation geometry and non-additive feature effects.

class-separation measure: it ranks neurons by observed behaviour rather than by directly inspecting the weights leading into or out of the layer.

layer_pruning_analysis(). Contribution scores alone do not prove that a neuron is necessary for the final prediction, so the pruning experiment tests their functional importance. Neurons are ordered from least to most contributive, and an activation mask progressively sets the least-contributive outputs to zero. Instead of altering the original network weights, the code constructs a “tail model” containing all layers after hidden layer 2 and feeds the masked activation vectors into this tail. Classification accuracy is measured with four, three, two, one and finally zero retained neurons. If accuracy remains nearly unchanged after a removal, the discarded signal was largely replaceable; a strong decrease indicates that the removed neurons carried information needed by the downstream classifier. Because hidden units interact, the accuracy curve need not decrease monotonically after every individual removal.

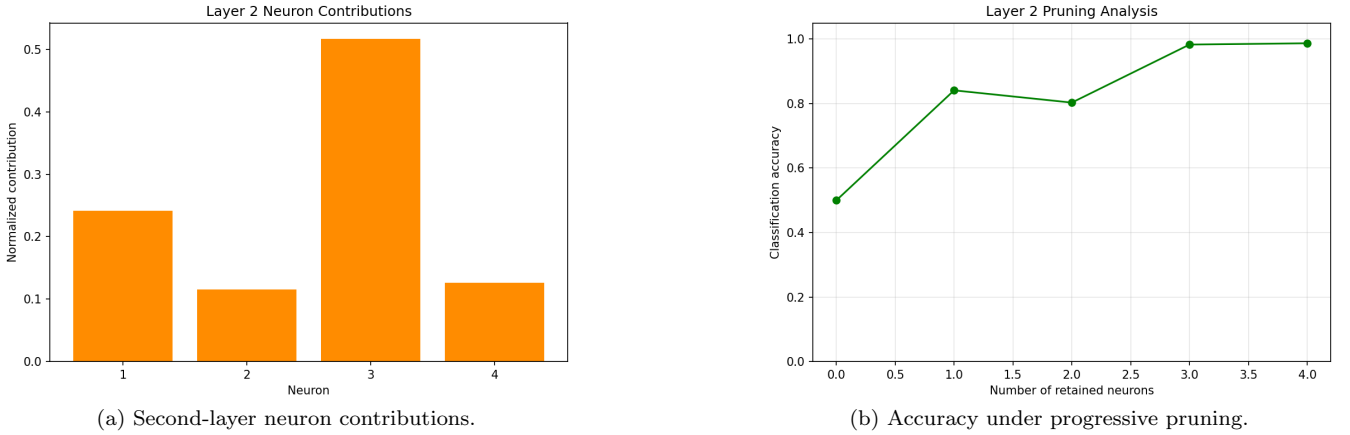


Figure 6: Neuron-level contribution and pruning analysis.

4.4 Observed Results

The full processed test matrix has shape (10000, 20) and the balanced analysis matrix (1000, 20); test accuracy is 0.9946. The top-10 PCA features are [1, 3, 5, 11, 2, 6, 14, 13, 8, 4], with Feature 1 contributing about 25.36% of normalized permutation importance. No dead neurons or neuron pairs above the 0.95 redundancy threshold are found. The silhouette score improves from 0.26363 in layer 1 to 0.46926 in layer 2, and the strongest first-layer input associations are [4, 3, 3, 7, 2, 1, 5, 1].

The maximum feature-interaction score is 0.10213; leading pairs are (1, 5): 0.10213, (1, 3): 0.09985, (1, 2): 0.09250, (1, 11): 0.07028, and (2, 3): 0.06985, confirming clear non-additive effects. Second-layer neurons rank [3, 1, 4, 2] from most to least contributive, giving pruning order [2, 4, 1, 3]. Accuracies for [4, 3, 2, 1, 0] retained neurons are [0.987, 0.983, 0.803, 0.841, 0.5]; with all second-layer activations removed, performance falls to balanced-set chance level.

5 Conclusion and Repository

Problem 1 recovers both affine regressors to floating-point precision; Problem 2 reconstructs the logistic classifier with perfect boundary agreement and no meaningful interaction; and Problem 3 reveals clearer hidden-layer class separation, feature interactions, and unequal neuron importance. All code and supporting material are available at github.com/dakshrathi-india/reverse_engg_lab_assign1.